COMP9414 Artificial Intelligence

Topic exercises: STRIPS Planning with ai9414

UNSW Sydney

1 Instructions

These exercises practise the planning ideas from the STRIPS tutorial: symbolic states, grounded actions, preconditions, add effects, delete effects, PDDL reading, and breadth-first planning.

You should already have completed the setup from the planning tutorial. If not, create a Python environment and install the package:

```
python3 -m venv .venv
source .venv/bin/activate
python -m pip install --upgrade pip
python -m pip install "ai9414>=0.1.4"
```

On Windows PowerShell, use:

```
py -3.11 -m venv .venv
.\.venv\Scripts\Activate.ps1
python -m pip install --upgrade pip
python -m pip install "ai9414>=0.1.4"
```

Check the demo list, then run the planning demo:

```
ai9414 list
ai9414 demo strips --example canonical_delivery
```

These exercises assume `ai9414` version `0.1.4` or newer. The office-delivery `strips` demo is the required demo for these exercises. The `blocksworld` demo is available as an optional extra test of the same Python planner interface.

Syntax reference. Use the STRIPS and PDDL appendices in the planning tutorial handout when you need to check how variables, grounded actions, preconditions, add effects, and delete effects are written.

2 Exercise 1: Validate and Repair a Plan

Consider this proposed plan for `canonical_delivery`:

```
1. move(robot, corridor, mail_room)
2. pickup_parcel(robot, parcel, mail_room)
3. move(robot, mail_room, corridor)
4. move(robot, corridor, office_a)
5. pickup_keycard(robot, keycard, office_a)
6. unlock_door(robot, keycard, lab_door, corridor, lab)
7. move(robot, corridor, lab)
8. drop_parcel(robot, parcel, lab)
```

1. Find the first action in the proposed plan that is not applicable when reached.
2. Name the missing precondition or false fact that makes it illegal.
3. Give one valid repaired plan. Try to keep it short, but you do not need to prove that it is the shortest repair.
4. In your repaired plan, identify the step where `locked(lab_door)` is deleted and `unlocked(lab_door)` is added.
5. Does your repaired plan achieve `at(parcel, lab)`? Explain how the final action makes the goal true.

3 Exercise 2: PDDL Reading and Writing

Read the following PDDL-style action schema:

```
(:action drop_parcel
  :parameters (?r ?p ?room)
  :precondition (and
    (at ?r ?room)
    (carrying ?r ?p)
  )
  :effect (and
    (not (carrying ?r ?p))
    (at ?p ?room)
    (handempty ?r)
  )
)
```

1. Translate this action into a STRIPS action schema with preconditions, add effects, and delete effects.
2. Ground the action for dropping the parcel in the lab.
3. Suppose the state contains `at(robot, lab)` and `carrying(robot, parcel)`. Write the facts changed by applying the grounded action.
4. In the `canonical_delivery` PDDL problem, list the facts that are static map facts.
5. Explain the difference between the lab-door map fact and `unlocked(lab_door)`. Why are both needed in the PDDL model?

4 Exercise 3: Complete and Test the Python Planner

Download the Python stub from the `strips` demo, open `solve_strips.py`, and complete `solve_planner(problem)` using forward BFS.

1. Use the helpers imported from `ai9414.strips` to get the initial facts, enumerate applicable actions, and apply grounded action strings.
2. Return a dictionary with `algorithm`, `status`, and `plan`. You may include `stats`, but they are optional.
3. Run `python solve_strips.py`.

4. In the browser, switch to `live Python`, click `Solve with Python`, and replay your plan.

5. Include one screenshot of the replay and briefly describe how you know the plan is being replayed from your Python solver.

5 Exercise 4: The Sussman Anomaly

This exercise uses a small blocks-world problem. There are three blocks, `A`, `B`, and `C`. A robot hand can hold at most one block. The predicate `clear(X)` means that no block is on top of `X`. This is also available in the optional demo as `ai9414 demo blocksworld --example sussman_anomaly`.

```
Initial state:                  Goal state:

    A                               A
    C     B                         B
------------- table                 C
                                 ------- table
```

The initial facts are:

```
on(A, C)
on(C, table)
on(B, table)
clear(A)
clear(B)
handempty
```

The goal is:

```
on(A, B)
on(B, C)
```

Use these action schemas:

```
pickup(?x)
  pre:  on(?x, table), clear(?x), handempty
  add:  holding(?x)
  del:  on(?x, table), clear(?x), handempty

putdown(?x)
  pre:  holding(?x)
  add:  on(?x, table), clear(?x), handempty
  del:  holding(?x)

unstack(?x, ?y)
  pre:  on(?x, ?y), clear(?x), handempty
  add:  holding(?x), clear(?y)
  del:  on(?x, ?y), clear(?x), handempty

stack(?x, ?y)
  pre:  holding(?x), clear(?y)
  add:  on(?x, ?y), clear(?x), handempty
  del:  holding(?x), clear(?y)
```

1. Write the initial state and goal as sets of facts.
2. Check whether `on(A, B)` can be achieved first by the plan:

   ```
   unstack(A, C)
   putdown(A)
   pickup(A)
   stack(A, B)
   ```

   Which goal fact is achieved, and which goal fact is still missing?
3. From the state after that four-action plan, explain why `pickup(B)` is not applicable.
4. What action would make `B` clear again? Which already-achieved goal fact would that action delete?
5. Find a valid plan that achieves both goal facts. Hint: try making `on(B, C)` true before `on(A, B)`.
6. Write a short explanation of why this is called an anomaly for planners that solve one goal completely before starting the next.
7. Write the `:init` and `:goal` sections of a small PDDL problem for this instance. You do not need to write the full domain.

6 Exercise 5: Modelling Faults

Planning models can be wrong even when the search code is correct. For each change below, explain what unrealistic or unwanted behaviour could follow. Answer using facts and actions, not just the picture.

1. Remove the delete effect `handempty(?r)` from `pickup_parcel(?r, ?p, ?room)`.
2. Remove the precondition `locked(?door)` from `unlock_door(?r, ?k, ?door, ?here, ?there)`.
3. Remove the add effect `handempty(?r)` from `drop_parcel(?r, ?p, ?room)`.
4. Remove `door_between(lab_door, corridor, lab)` from the initial facts but keep `unlocked(lab_door)` achievable.
5. Remove the goal fact `at(parcel, lab)` and replace it with `at(robot, lab)`. What would the planner now optimise for?